ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 2

ROBOTICS FUNDAMENTALS

PRACTICE EXAM 6 (PAPER 2)

MARKS: 80 **TIME: 4 hours**

NC(V) Programming and Robotics, Level 2. Laid out like the DHET second paper (Raspberry Pi practical, 80 marks, 4 hours): a running-light arrow with breadboard and flowchart, completing and commenting a plant-watering program, and a fire-alarm project with breadboard, Python code and a user manual.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 2. It is not a Department of Higher Education and Training examination paper.

INSTRUCTIONS AND INFORMATION

1. This question paper consists of THREE questions. Answer ALL the questions.
2. These questions are done on a computer, using draw.io (or similar) for flowcharts, Fritzing for breadboard circuits, a Python 3 IDE for code and a word processor for documents. If you do not have a computer, draw and write your answers neatly by hand.
3. Use a Raspberry Pi (any model with a 40-pin header, or a Pi Pico). Use the BCM (GPIO) numbers of the pins.
4. Put your name or examination number on every printout or page. Save your work regularly.

QUESTION 1

> A shopping centre wants a sign with a "running" arrow that points the way to the parking exit. Six red LEDs are placed in a row. They must light up one after the other from left to right, each for 0,3 seconds, and the pattern must repeat until the program is stopped.

Components:

- Raspberry Pi
- 6 × red LEDs
- 6 × 220 Ω resistors
- Breadboard
- Jumper wires

1.1 Using the components above, create a breadboard circuit for the running arrow (use Fritzing). Each LED must be connected to its own GPIO pin through its own resistor, with all cathodes going to GND. (10)

1.2 Create a flowchart of the logic you would use to make the LEDs run from left to right and repeat. (11)

1.3 Save and print (or neatly present) the breadboard circuit and the flowchart. (2)

[23]

QUESTION 2

> A soil-moisture sensor module is pushed into the soil of a pot plant. Its digital output (DO) is connected to GPIO 4 and goes HIGH when the soil is dry. When the soil is dry, a red LED on GPIO 22 must switch on and a buzzer on GPIO 23 must beep for half a second. When the soil is moist, the LED must be off. The soil is checked every 5 seconds.

Below is the program with ELEVEN parts missing, marked …(1)… to …(11)…. Analyse the code and answer the questions.

```
import RPi.GPIO as GPIO
import time

SENSOR = 4
```

```
LED = 22
BUZZER = 23

GPIO.setmode(GPIO.…(1)…)
GPIO.setup(SENSOR, GPIO.…(2)…)
GPIO.setup([LED, BUZZER], GPIO.…(3)…)

try:
    while …(4)…:
        if GPIO.input(SENSOR) == GPIO.…(5)…:
            print("Soil is dry - water the plant!")
            GPIO.output(LED, GPIO.…(6)…)
            GPIO.output(BUZZER, GPIO.HIGH)
            time.sleep(0.5)
            GPIO.output(BUZZER, GPIO.…(7)…)
        else:
            print("Soil is moist")
            GPIO.output(LED, GPIO.…(8)…)
        time.…(9)…(5)
except …(10)…:
    print("Program stopped")
finally:
    GPIO.…(11)…()
```

2.1 Open a Python IDE and create a new file called Question2_YourName.py. (2)

2.2 Recreate the program, filling in all the missing parts so that it works. (11)

2.3 Add comments to the code that explain what it does, including the pin set-up, the loop, the decision and the exception handling. (10)

2.4 Save and print (or neatly present) the code. (2)

[25]

QUESTION 3

Design a fire alarm for a college workshop using a Raspberry Pi. A flame sensor module's digital output goes LOW when it detects a flame. While there is no flame, a green LED is on to show that the system is working. When a flame is detected, the green LED goes off, a red LED flashes and a buzzer sounds. The alarm keeps going until a teacher presses the RESET button, even if the flame disappears.

You have the following components:

- 1 × Raspberry Pi
- 1 × flame sensor module (VCC, GND, DO)
- 1 × red LED and 1 × green LED
- 2 × 220 Ω resistors
- 1 × active buzzer
- 1 × push button (RESET)
- Breadboard and jumper wires

3.1 Using the components above, create a breadboard circuit for the fire alarm (use Fritzing). (10)

3.2 Open a Python IDE and create a new file called Question3_YourName.py. (2)

3.3 Write the Python code for the fire alarm. (10)

3.4 Write a short user manual for the fire alarm. It must include: the name and a description of the system; basic operation; troubleshooting; and frequently asked questions (FAQs). (10)

[32]

TOTAL: 80

END OF QUESTION PAPER